

**TF****TERRAFLY**

TerraFly Source-to-System Atlas

Problem, evidence states, complete request traces, source atlas, artifacts, claims and live proof

Verified against the packaged TerraFly project and live application on 2026-08-28.

Scientific position

Measured mountains come from a named DEM. Photo AI begins as relative shape. AI-derived metric files exist only after independent validation passes. Good-looking 3D is never substituted for evidence.

Contents

Contents	2
TerraFly Source-to-System Atlas	4
1. The one-minute truth	4
2. Exact SIH/ISRO problem identity	4
3. Existing approaches and the gap TerraFly addresses	5
4. Feasibility and impact	6
5. Scientific states: evidence, not a visual switch	6
6. Workflow one: DEM Terrain	7
7. Workflow two: Photo AI	8
8. Metric calibration: how metres unlock	9
9. The 3D system from pixels to interaction	10
10. Five exact request flows	11
11. File-by-file atlas	13
12. Runtime artifact atlas	15
13. Security, privacy, licensing and repeatability	16
14. Verification record for this atlas	17
15. Judge demonstration and wording	17
16. Known inconsistencies and next decisions	18
17. Recreate TerraFly in the right order	18
TerraFly claims ledger	19
TerraFly inconsistencies and implementation limitations	21
Non-negotiable scientific limitations	22
TerraFly source and line index	23
Runtime and scientific contract	23
AI inference	23
Artifacts and 3D	23
Frontend	24
Launch, verification and demonstration	24
Reading order for a complete beginner	25
Visual evidence and editable diagrams	26
TerraFly runtime architecture	26
Scientific states require new evidence	26
DEM Terrain workflow	27
Photo AI workflow	27

Metric calibration gate 28

Height pixels to textured mesh and measurement 28

Five auditable request flows 29

01 Live Start Screen 30

02 Live Dem Terrain Result 31

03 Live Photo Ai Calibrated 32

04 Live Calibration Passed 33

TerraFly Source-to-System Atlas

Project: TerraFly / SIH 2026 problem 26175 Verified build: TerraFly_FINAL_WINDOWS Audit date: 2026-08-28

Purpose: make a beginner able to operate, trace, defend and recreate the actual system.

The evidence rule used throughout this atlas is: source location + observed/tested behavior + scientific limitation + safe judge wording.

1. The one-minute truth

TerraFly is a local terrain workbench with two deliberately separate workflows.

1. DEM Terrain - recommended: combine georeferenced optical imagery with a named metre-valued DEM. The DEM supplies geometry and elevations; the imagery supplies color. This is the strongest current mountain demonstration.
2. Photo AI - experimental: run the real Depth Anything V2 Large monocular model on one image. The first result is a relative 0-to-1 surface, never metres. A georeferenced input can become a metric DSM-like result only when an independent aligned reference DSM or separate GCP evidence passes the calibration gates.

This separation is the core scientific decision. A good-looking mesh is not automatically measured terrain. [Code: backend/terrafly/schemas.py:9-13; backend/terrafly/main.py:49-204 | Tests: backend/tests/test_geotiff.py:32-86; backend/tests/test_calibration.py:68-241 | Limitation: metric truth depends on source DEM or independent vertical evidence | Judge: "TerraFly distinguishes supplied elevation, relative AI shape and calibrated metric output instead of presenting them as the same claim."]

2. Exact SIH/ISRO problem identity

Problem ID: SIH26175 Organization: Indian Space Research Organisation (ISRO) Exact title: DepthWizard - Single-View Height Estimation and 3D Flythrough

The public SIH listing describes this problem: DEM/DSM data is important for urban planning, disaster management and military applications; conventional stereo, LiDAR and InSAR can be costly, sensor-specific or computationally demanding. The requested system accepts optical RGB remote-sensing imagery, estimates elevation using a pretrained monocular backbone, produces relative outputs for ordinary images and absolute DSM GeoTIFFs when geospatial and vertical reference evidence is supplied, textures a 3D mesh, and supports interactive flythrough plus height/slope inspection. Evaluation is split between DSM accuracy across multiple landscapes and visualization/UX. The expected result is a unified, documented suite producing standard geospatial DSM outputs and accessible 3D exploration.

Verification locators: official SIH portal <https://sih.gov.in/sih2026PS#ViewProblemStatement26175>; public cross-check <https://sih-fit.vercel.app/problem/SIH26175>; secondary cross-check <https://sih-buddy.vercel.app/ps/SIH26175>. The official portal is authoritative; mirrors were used only to cross-check the publicly displayed text.

2.1 Users, pain, root cause and stakeholders

Element	TerraFly interpretation
Primary users	GIS analysts, planners, disaster-research teams, remote-sensing students and technical evaluators who need an inspectable terrain result without building a bespoke ML/3D stack.
Pain	Imagery is visually rich but does not directly expose height; elevation products and viewers require GIS knowledge; monocular AI often produces attractive but uncalibrated shape.

Element	TerraFly interpretation
Root cause	A single 2D projection loses absolute scale and offset. CRS adds horizontal location, not a vertical datum. Sensor/domain differences introduce spatially varying errors.
Stakeholders	ISRO/SIH evaluators, data providers, local authorities, researchers, emergency planners, defense analysts, software operators and people affected by decisions derived from terrain data.
Risk owner	The team owns software behavior and claim wording. The data provider owns source DEM accuracy. A deployment organization owns operational validation and policy.

3. Existing approaches and the gap TerraFly addresses

Existing approach	Strength	Gap for this project	TerraFly position
Global DEM products such as SRTM	Broad coverage, metric elevation and known provenance	Native resolution, datum and surface/terrain meaning may not match a local optical image; not a textured interactive product by itself	DEM Terrain aligns a named source, preserves values and records both grids. It never claims resampling creates detail.
Photogrammetry	Strong geometry from overlapping images and camera models	Needs multiple overlapping views, calibration and processing	TerraFly targets a single-view workflow but accepts lower certainty and requires external vertical evidence for metres.
LiDAR	High-quality direct range measurements and point clouds	Cost, coverage, acquisition and processing burden	TerraFly can visualize derived elevation but does not replace LiDAR accuracy.
InSAR/stereo satellite processing	Can recover elevation over large regions	Sensor/geometry expertise, coherent pairs or stereo acquisitions and expensive pipelines	TerraFly offers a simpler local front end around supplied DEM or a monocular baseline.
Google Earth/Bhuvan-style visualization	Excellent global exploration and polished 3D	It is a visualization/data service, not a reproducible user-owned inference pipeline; captured Google Earth output cannot be used to reconstruct 3D under the cited Geo Guidelines	TerraFly exposes source files, arrays, calibration reports and hashes.
Desktop GIS such as QGIS	Comprehensive raster inspection, reprojection and analysis	Learning curve and no built-in TerraFly AI/calibration/GLB workflow	TerraFly narrows the task to upload, evidence gate, 3D inspection and export; QGIS remains essential for preparing real data.
Monocular-depth models	One-image relative structure with pretrained checkpoints	Scale ambiguity, ordinary-camera domain bias, global tilt and no automatic vertical datum	Photo AI labels the result relative, records the model and exposes a separate evidence gate.

3.1 What is innovative and what is reused

Existing technology reused: FastAPI, Pydantic, Pillow, Rasterio, NumPy, PyTorch, Transformers, Depth Anything V2, React, TypeScript, Three.js, glTF/GLB, pytest and Vitest. The model and formats are not claimed as inventions.

[Source: `THIRD_PARTY_NOTICES.md:1-24`; `pyproject.toml`; `frontend/package.json` | Evidence: production build and test suite | Limitation: upstream licenses and domain limits apply | Judge: "We integrated established open tools and a real pretrained baseline into an auditable local workflow."]

TerraFly engineering contribution: the explicit scientific state machine; two honest workflows; raw/canonical/display separation; output-direction regression; bounded affine-aligned tiling; exact-grid independent calibration gates; locked

metric artifacts; texture/normal-aware GLB; point measurements; per-job hashes and manifests; local release verification. These are integration and evidence-design contributions, not a new foundation model.

4. Feasibility and impact

4.1 Engineering feasibility

- Demonstrated: packaged backend 40/40 tests; frontend 10/10 tests; frontend production build; real CUDA inference; DEM Terrain smoke; calibration oracle; browser interaction; zero browser console errors during the audited flows.
- Resource shape: a configured 25 MB upload limit, 50 million pixel limit and 768 MiB processing estimate; tiling defaults of 1024 with 128 overlap and a 256-tile cap. [Code: config.py:11-31; imaging.py:45-59; inference/tiling.py:8-99 | Tests: test_api.py:86-121; test_tiling.py:9-64 | Limitation: time and GPU memory still scale with scene size | Judge: "The runtime uses explicit size and tile bounds rather than accepting unbounded inputs."]
- Local deployment: prebuilt frontend and API share a loopback address. The first setup and uncached checkpoint require internet and several gigabytes; after caching, scene data stays local. [Code: main.py:233-239; scripts/start.ps1:1-115 | Evidence: live package at 127.0.0.1:8012 | Limitation: not a zero-install air-gapped archive | Judge: "It runs locally after one-time dependency and model setup."]

4.2 Scientific feasibility

- Proved: deterministic preservation/alignment of a supplied metric DEM; explicit relative output; robust positive affine calibration; independent validation and fail-closed artifacts.
- Not proved: real Photo AI height accuracy across urban, sparse, hilly and forested data. A global affine fit cannot cure local model errors.
- Best current scientific demo: a real, licensed optical GeoTIFF and matching DEM, visibly attributed, in DEM Terrain.

4.3 Deployment feasibility

The current package is appropriate for a controlled judge laptop or lab workstation. It is not an operational cloud service: there is no authentication, multi-user scheduler, remote storage, monitoring service-level objective or completed production security review. Bind-to-loopback and path checks reduce local risk but do not establish enterprise readiness.

4.4 Demonstrated, potential and unproved impact

Level	What may be said
Demonstrated	TerraFly turns a supplied optical+DEM pair into auditable textured metric terrain; runs a real monocular baseline; locks metric AI outputs until evidence passes; exports arrays, reports, GeoTIFF and GLB; supports interactive comparison.
Potential	Faster terrain briefing, education, preliminary visual inspection, source alignment checking and a foundation for future flood/slope/line-of-sight modules.
Not demonstrated	Disaster prediction, military readiness, survey-grade accuracy, operational reliability, real Himalayan accuracy, general urban building-height accuracy or commercial licensing suitability.

5. Scientific states: evidence, not a visual switch

PNG/JPG input -----> Relative
0..1, no CRS, no metres

GeoTIFF image -----> Georeferenced Relative

```
0..1 + horizontal CRS/transform  
still no vertical scale/datum
```

```
Georeferenced Relative + independent reference/GCP gate PASS  
-> Metric Calibrated  
AI-derived metres + datum evidence
```

```
Optical GeoTIFF + named metre DEM ---> Metric Source DEM  
metres inherited from source DEM
```

The schema encodes four labels because the two metric states have different evidence origins. [Code: schemas.py:9-13 | Test: test_api.py:94-112; test_geotiff.py:32-86 | Limitation: a label cannot validate dishonest evidence | Judge: "Metric Source DEM means supplied elevation; Metric Calibrated means an AI surface transformed only after independent validation."]

Relative: normalized float32 0-to-1 shape. No metres. Georeferenced Relative: same vertical ambiguity plus CRS, affine transform, bounds and horizontal pixel size. Metric Source DEM: the named DEM already supplies metre values and datum/source declaration. Metric Calibrated: an AI surface receives a positive affine scale/offset only after gates pass.

6. Workflow one: DEM Terrain

6.1 Step-by-step

1. Optical GeoTIFF validation. The upload is chunk-limited; safe names and extensions are checked; Rasterio inspects bands, shape, CRS, affine transform, bounds, masks and pixel count. [Code: imaging.py:31-137; main.py:69-136 | Test: test_api.py:81-121; test_terrain.py:65-137 | Limitation: syntactic validity does not prove imagery quality | Judge: "We reject unsafe, corrupt, oversized or ungeoreferenced inputs before processing."]
2. DEM validation. TerraFly requires one band, CRS, finite valid values, a recognized metre unit or an explicit metre source declaration, and enough valid pixels. [Code: terrain.py:34-77 | Tests: test_terrain.py:139-176 | Limitation: values can still be inaccurate or use a wrong declared datum | Judge: "We validate the raster contract; the named provider remains the elevation authority."]
3. CRS/transform/overlap inspection. Optical and DEM bounds are transformed into a common spatial comparison. Insufficient overlap fails before metric output. [Code: terrain.py:84-145 | Test: test_terrain.py:139-155 | Limitation: overlap does not prove temporal or semantic match | Judge: "Both files must cover the same place; a filename match is not enough."]
4. Reprojection and resampling. Rasterio reprojects the DEM onto the optical grid using bilinear resampling while preserving NoData. The alignment report records original/output grid and the method. [Code: terrain.py:84-145 | Test: test_terrain.py:65-137 | Limitation: a 30 m DEM resampled to 10 m remains 30 m information | Judge: "Alignment changes the grid, not the source resolution or accuracy."]
5. Metric-array preservation. The aligned metre array is saved as aligned_source_dem.npy and metric_surface.npy; invalid cells remain masked/NoData. [Code: terrain.py:171-192,213-348 | Test: test_terrain.py:65-137 | Limitation: inherited source uncertainty is not estimated | Judge: "The scientific array remains in source metres; display processing is separate."]
6. Display normalization. Valid metre values are robustly mapped for mesh height/color only. [Code: terrain.py:148-168 | Test: test_terrain.py:65-137 | Limitation: visual exaggeration can mislead without the legend | Judge: "Normalization and exaggeration affect display, never the saved elevations."]
7. Mesh and texture. A grid of height pixels becomes vertices and triangles; the optical image becomes a texture. UVs place image pixels on the mesh, normals support lighting and PBR material. [Code: artifacts.py:83-296; terrain.py:213-348 | Test: test_artifacts.py:31-128 | Limitation: a height field has one height per x/y and cannot model overhangs | Judge: "Geometry comes from the DEM and color"]

comes from the optical image."]

8. Measurement. A/B clicks sample the metric analysis grid; projected metre CRS enables horizontal distance and slope. [Code: surfaceGeometry.ts:122-200; SurfaceViewer.tsx:262-317; App.tsx:155-186 | Test: SurfaceViewer.test.ts:15-142 | Limitation: sampled/downsampled viewer values are inspection values, not survey certification | Judge: "The original metric arrays remain the scientific export; clicks support interactive comparison."]
9. Evidence exports. The job contains the aligned source, metric arrays/GeoTIFF, alignment report, display grids, texture, GLB, diagnostics, source copies and SHA-256 manifest. [Code: terrain.py:171-348; artifacts.py:18-33 | Test: test_terrain.py:65-137; terrain smoke PASS | Limitation: hashes prove byte identity, not correctness | Judge: "The bundle records exactly which input and derived bytes produced the scene."]

6.2 The mountain answer

Mountains generally look better through DEM Terrain because geometry comes from measured/sourced elevation rather than image brightness. Snow, shadow and ridge contrast remain texture, not height evidence. The bundled mountain pair is synthetic and demonstrates software only. A judge-ready real scene must use licensed matching optical imagery and DEM with visible source, datum, bounds and attribution. [Docs: docs/REAL_TERRAIN_DATA_GUIDE.md:5-51; docs/DEMO_SCRIPT.md:21-59 | Evidence: live screenshot 02 | Limitation: no real Himalayan pair is bundled | Judge: "This scene demonstrates the workflow; source accuracy belongs to the cited DEM."]

7. Workflow two: Photo AI

7.1 Input to canonical relative surface

1. Validation and preprocessing. PNG/JPG/TIFF are decoded safely; a GeoTIFF retains spatial metadata; a single band is repeated into RGB with an out-of-domain warning; image modes are normalized to three channels. [Code: imaging.py:140-175; pipeline.py:18-48 | Test: test_api.py:16-92 | Limitation: three identical thermal channels do not make an optical model scientifically valid | Judge: "Single-band execution proves software compatibility only."]
2. Model loading. The normal adapter lazily loads depth-anything/Depth-Anything-V2-Large-hf, resolves the exact checkpoint revision and caches it. [Code: depth_anything_v2.py:11-73; factory.py:9-23 | Evidence: verified revision 7581137... and CUDA | Limitation: CC-BY-NC-4.0 checkpoint and general-image training restrict use/claims | Judge: "Every job records the real model revision; it is a relative baseline, not our trained overhead model."]
3. CUDA and fallback. The adapter selects CUDA when available, uses autocast, and catches CUDA out-of-memory to clear memory, move to CPU and retry. [Code: depth_anything_v2.py:74-158 | Evidence: real CUDA smoke | Limitation: CPU fallback is slower and was not benchmarked for every input size | Judge: "The device and fallback warning are recorded in the job."]
4. Raw prediction. The untouched model field is saved as raw_model_output.npy. [Code: pipeline.py:44-76; artifacts.py:735-888 | Test: test_api.py:16-79 | Limitation: raw values have arbitrary scale/offset | Judge: "We retain the raw field so the conversion can be audited."]
5. Direction convention and historical repair. Depth Anything V2 relative output behaves as inverse depth/proximity, where larger/nearer is mapped to higher surface in a near-nadir scene. The earlier extra inversion made roofs holes. The convention is now explicit and normalized once. [Code: conventions.py:9-98 | Tests: test_adapter_contract.py:24-55; test_artifacts.py:130-166 | Limitation: near-nadir assumption and ordering test do not establish metric truth | Judge: "A synthetic raised-roof regression prevents the repaired direction bug from returning."]
6. Robust normalization. Global 2nd and 98th percentiles limit outlier influence; the canonical float32 array becomes 0-to-1. [Code: conventions.py:29-98 | Test: test_adapter_contract.py:47-55 | Limitation:

relative contrast can vary by scene | Judge: "The canonical result is relative by design and is never labelled metres."]

7.2 Large images and tiles

When the trigger is exceeded, TerraFly makes an overlapping bounded tile grid. Neighboring crops can have different arbitrary depth scale and offset, so each tile is aligned to accumulated overlap by a positive affine relation. Feather weights blend edges, and the complete raw field is normalized once. [Code: `depth_anything_v2.py:94-137`; `tiling.py:8-99` | Tests: `test_tiling.py:9-64`; script: `smoke_tiled_model.py` | Limitation: overlap agreement can propagate model bias and does not create absolute scale | Judge: "Tiling is a memory/detail engineering method, not an accuracy claim."]

7.3 Canonical versus display artifacts

`relative_surface.npy` is immutable scientific input for later calibration. A copy becomes `display_grid.json` after isolated-spike suppression, RGB-guided smoothing, conservative roof flattening and an extreme-delta safety cap. [Code: `artifacts.py:299-519,735-888` | Test: `test_artifacts.py:168-210` | Limitation: cleanup can make shapes look better without making them truer | Judge: "Display cleanup never changes canonical arrays or metric measurements."]

The GLB embeds the real source texture, UVs, smooth normals, triangle indices and a lit PBR material. Steep faces use neutral shading because a top-down image contains no genuine facade pixels. [Code: `artifacts.py:83-296` | Test: `test_artifacts.py:31-128` | Limitation: neutral walls are visual honesty, not recovered facade data | Judge: "We avoid stretching roof pixels down steep walls."]

8. Metric calibration: how metres unlock

The model surface is converted by $\text{elevation}_m = \text{scale} * \text{relative} + \text{offset}$. TerraFly fits this robustly and requires $\text{scale} > 0$. Two evidence modes exist.

- Reference DSM: same CRS, width, height and affine transform as the Photo AI input. A spatial checkerboard is reserved before fitting.
- GCP: control points fit the transform; separate validation points evaluate it. The API supports this path; the current UI exposes the reference-raster path.

[Code: `calibration.py:54-118,348-532`; `main.py:157-204` | Tests: `test_calibration.py:68-241` | Limitation: users can still supply non-independent or inaccurate evidence | Judge: "Controls determine scale and offset; held-out evidence decides whether metric files exist."]

8.1 Exact gates

Gate	Rule in code	Why	Rejection behavior
Positive scale	fitted scale must be greater than zero	Prevents a direction-flipped metric surface	report saved; metric files absent
Relative span	control relative span at least 0.05	Avoids fitting metres to nearly flat/no-information predictions	same
Inlier ratio	at least 0.75	Robust fit must explain most controls	same
Spatial coverage	at least 40% of both x and y axes	Prevents clustered controls from pretending to represent the scene	same
Held-out evidence	sufficient independent validation samples	Training error cannot approve itself	same

Gate	Rule in code	Why	Rejection behavior
Held-out RMSE	at or below user-declared threshold	Connects pass/fail to a visible tolerance	same
R-squared	at least 0.50	Requires the affine result to explain meaningful variation	same

The gate logic is centralized in `calibration.py:136-163`; pass-only writing is in `calibration.py:276-345`. Poor evidence creates `calibration_report.json` and residual diagnostics but keeps `metric_surface.npy/.tif` locked. Tests cover pass, mismatch, rejection, GCP pass and NoData preservation at `test_calibration.py:68-241`.

8.2 Why the bundled near-zero RMSE is not accuracy

The bundled reference is deliberately generated from the model output using approximately $40 * \text{relative} + 100$, with training-only outliers. The verified run recovered scale about 40 and offset about 100 with held-out RMSE about 0.000002 m. That proves array orientation, split isolation, robust fitting, gate logic, metric file writing, hashes and UI state. It cannot prove performance on an independent landscape because the "truth" was constructed from the prediction. [Evidence: `sample_data/terrafly_calibration_demo_metadata.json`; verified 18-artifact CUDA job | Limitation: synthetic oracle | Judge: "Near-zero error belongs to the software oracle and is excluded from our model metric card."]

9. The 3D system from pixels to interaction

```

height pixel (row, col)
-> vertex (x, displayed height, z)
-> two triangles per grid cell using index triples
-> UV coordinate maps the optical texture
-> neighboring triangles produce a smooth unit normal
-> PBR material + movable directional light
-> camera controls and raycaster
-> point A/B sample analysis or metric grid

```

Concept	TerraFly implementation	Limitation / judge wording
Height pixels	Relative, display or metric grids carry one value per raster cell. <code>surfaceGeometry.ts:202-255</code>	One height per x/y; no overhangs.
Vertices and indices	A regular grid is built into indexed triangles. <code>artifacts.py:112-296</code> ; <code>surfaceGeometry.ts:202-255</code>	Mesh may be downsampled for responsiveness.
Normals	Neighbor gradients create unit normals for lighting. <code>artifacts.py:83-98</code>	Lighting reveals shape but is not evidence.
UV and texture	UV coordinates map embedded <code>texture.png</code> onto the surface. <code>artifacts.py:112-296</code>	Steep faces do not have source facade pixels.
PBR material	Lit material responds to movable sun. <code>artifacts.py:112-296</code> ; <code>SurfaceViewer.tsx:112-260</code>	It is a visual material, not reflectance calibration.
Neutral steep faces	Strong slopes use neutral shading. <code>artifacts.py:112-296</code>	Avoids invented/stretched texture; it is not reconstructed wall color.
Orbit	left drag rotate, right drag pan, wheel zoom. <code>SurfaceViewer.tsx:112-220</code>	Inspection control only.
First-person	pointer-lock look, WASD, Q/E and Shift. <code>SurfaceViewer.tsx:318-344</code>	Free flight; no collision/gravity.

Concept	TerraFly implementation	Limitation / judge wording
Photo/height colors	Separate material/color modes and numeric legend. SurfaceViewer.tsx:45-110,389-440; surfaceGeometry.ts:32-64	Height colors use relative or metric units exactly as labelled.
Wireframe and lighting	toggles expose topology and normals. SurfaceViewer.tsx:45-220	Visual QA, not quantitative accuracy.
Exaggeration	display-only vertex multiplier. SurfaceViewer.tsx:45-110; surfaceGeometry.ts:202-255	Never changes .npy/GeoTIFF.
A/B sampling	raycaster maps intersection UV to analysis grid; bilinear sample returns relative/metres. SurfaceViewer.tsx:262-317; surfaceGeometry.ts:144-200	Output inherits grid/source uncertainty.
Distance and slope	projected CRS transform supplies horizontal metres, then rise/run. App.tsx:155-186; surfaceGeometry.ts:122-142	Disabled if horizontal units are not metres.
Structures	convex extrusions from raised connected candidates. artifacts.py:590-732; surfaceGeometry.ts:66-105	Nonsemantic visual layer; can contain trees or miss roofs.

10. Five exact request flows

Flow A - Upload button to Photo AI result

1. App.tsx:41-153,188-288 stores the selected file and starts submission.
2. api.ts:3-34 constructs multipart POST /api/jobs.
3. main.py:49-67 validates/saves input, creates a job and schedules the worker.
4. pipeline.py:18-96 inspects/preprocesses, selects the adapter and runs inference.
5. factory.py:9-23 chooses real or explicitly allowed test adapter; depth_anything_v2.py:11-158 predicts.
6. conventions.py:9-98 applies direction and canonical normalization.
7. artifacts.py:735-888 writes numeric, preview, texture, diagnostics, structures and GLB outputs.
8. jobs.py:28-63 updates job.json atomically; artifact hashes populate the manifest.
9. App.tsx:80-153 polls api.ts:36-46 until complete.
10. App.tsx:290-480 passes artifact URLs to SurfaceViewer.tsx:18-440.
11. Evidence: test_api.py:16-79, test_adapter_contract.py:24-55, App.test.tsx:10-190, verified real CUDA live run.

Failure prevented: deterministic test inference cannot be silently enabled. Limitation: output remains relative until external evidence. Judge: "The browser result can be traced through one typed route, one worker and a hash-bound artifact set."

Flow B - DEM upload to measured terrain

1. App.tsx:188-288 selects DEM Terrain and collects optical, DEM, source and datum.
2. api.ts:48-69 posts multipart data to /api/terrain-jobs.
3. main.py:69-136 validates both inputs and creates the job.
4. terrain.py:34-145 checks the DEM, overlap and alignment.

5. `terrain.py:148-192,213-348` preserves metric values, creates display data, GeoTIFF, report and mesh.
6. `jobs.py:28-63` persists progress and completion; `App.tsx:80-153` polls.
7. `SurfaceViewer.tsx:18-440` receives metric analysis grid and terrain mode.
8. Evidence: `test_terrain.py:65-176`, terrain smoke PASS, live screenshot 02.

Failure prevented: insufficient overlap and non-metre unit are rejected. Limitation: source DEM accuracy is not improved. Judge: "TerraFly makes a supplied elevation product inspectable and auditable; it does not call resampling super-resolution."

Flow C - Calibration request to passed or rejected result

1. `App.tsx:360-406` shows the reference DSM form for a georeferenced relative job.
2. `api.ts:36-46` posts the reference and declared RMSE limit to `/api/jobs/{id}/calibrate/reference`.
3. `main.py:157-184` validates job state and dispatches calibration.
4. `calibration.py:348-461` requires exact grid/CRS/affine, builds a checkerboard holdout and robustly fits controls.
5. `calibration.py:136-163` evaluates every gate.
6. `calibration.py:276-345` always writes the report; only a pass writes metric files and changes state.
7. `App.tsx:80-153,408-480` reloads/polls and renders Passed or Rejected evidence.
8. Evidence: `test_calibration.py:68-241`, verified live screenshots 03-04.

Failure prevented: training error cannot approve the reference workflow. Limitation: a globally good affine score can hide class/local errors. Judge: "Rejection is a valid evidence result; the product does not create metric files to satisfy the demo."

Flow D - Artifact link to downloaded file

1. `App.tsx:408-480` renders only artifacts present in the manifest.
2. `api.ts:71-73` builds `/api/jobs/{id}/artifacts/{name}`.
3. `main.py:206-219` validates job ID, resolves the artifact and checks containment/existence.
4. FastAPI returns the exact stored bytes; SHA-256 in the manifest enables verification.
5. Evidence: `test_api.py:16-79,123-131`; verified 18 artifact hashes.

Failure prevented: path traversal. Limitation: a matching hash proves identity, not scientific quality. Judge: "Downloads are bound to the selected job and each non-manifest artifact has a recorded digest."

Flow E - Viewer click to measurement

1. `SurfaceViewer.tsx:262-317` raycasts the mouse into the visible mesh.
2. The intersection UV maps to raster coordinates.
3. `surfaceGeometry.ts:144-200` bilinearly samples metric or relative analysis data.
4. `SurfaceViewer.tsx` sends A/B points to `App.tsx`.
5. `App.tsx:155-186` calculates vertical difference and, for projected metre CRS, distance and slope.
6. Evidence: `SurfaceViewer.test.ts:15-142`; live A/B interaction.

Failure prevented: display exaggeration is not used as the numeric value. Limitation: the analysis grid can be downsampled. Judge: "The click samples the analysis data associated with the mesh, not screen height or lighting."

11. File-by-file atlas

Read each row as a compact call graph: purpose; entry points; dependencies; input -> output; protected failure; evidence; beginner start.

11.1 Launch and backend

File	Atlas
Start-TerraFly.cmd	Double-click entry; calls <code>scripts/start.ps1</code> ; no data input -> launcher process; prevents a user from memorizing commands; verified by packaged live launch; read all 6 lines.
<code>scripts/start.ps1</code>	Checks environment/port and starts Uvicorn; called by CMD; calls PowerShell process/network helpers; project path -> local server/log; avoids attaching to unrelated port owner; live launch; read endpoint and port functions first.
<code>backend/terrafly/config.py</code>	Central limits/model/runtime paths; imported by routes/workers; environment -> Settings; prevents scattered unsafe defaults; API limit tests; read <code>Settings</code> .
<code>schemas.py</code>	Pydantic vocabulary for states/jobs/artifacts/calibration; called across API/worker/UI JSON; validated data -> serialized contract; prevents undocumented shapes; capability/API tests; start with <code>ScientificState</code> .
<code>jobs.py</code>	Per-job directory and atomic state store; called by routes/workers; job ID and updates -> <code>job.json</code> ; prevents path escape/partial writes; deletion security test; read <code>_job_dir</code> then save.
<code>imaging.py</code>	Safe upload and raster/image inspection; called by routes/workers; byte stream -> saved/decoded metadata; prevents oversized, corrupt and decompression-bomb input; API tests; read <code>save_upload</code> , <code>inspect_geotiff</code> , <code>load_normal_image</code> .
<code>main.py</code>	FastAPI composition and routes; called by Uvicorn; calls validators, workers and static files; HTTP -> jobs/files; prevents invalid state/path access; API tests; read <code>/health</code> , capabilities, then the five job/artifact routes.
<code>pipeline.py</code>	Photo AI orchestration; called by Photo route background task; calls imaging, adapter, conventions/artifacts/jobs; input image -> relative bundle; catches failure and records it; API and real smoke; read <code>run_pipeline</code> top to bottom.
<code>terrain.py</code>	DEM Terrain orchestration; called by terrain route; calls Rasterio alignment and artifact helpers; optical+DEM -> metric terrain bundle; prevents bad overlap/unit claims; terrain tests; read validation, alignment, then worker.
<code>calibration.py</code>	Robust affine fit, independent split/gates and pass-only metric export; called by calibration routes; relative+evidence -> report and optional metric bundle; prevents training-error metric claims; calibration tests; read <code>_gate</code> , reference and GCP functions.
<code>artifacts.py</code>	Hashes, grids, cleanup, GLB, diagnostics and artifact manifest entries; called by both workers/calibration; arrays+texture -> files; prevents canonical/display confusion and broken GLB contracts; artifact tests; read writer then follow helper calls.
<code>evaluation.py</code>	Masked real height metrics; called by benchmarking scripts; prediction+truth -> RMSE/MAE/bias/correlation; refuses mismatched/insufficient evidence; evaluation tests; read single evaluator.

11.2 Inference

File	Atlas
<code>inference/base.py</code>	Adapter interface and Prediction; factory/workers depend on it; image -> raw field/metadata; prevents incompatible adapters; adapter tests; read the dataclass and protocol.
<code>inference/factory.py</code>	Selects real vs test adapter; called by pipeline; settings -> adapter; prevents accidental test-only production use; contract test at line 57; read entire file.

File	Atlas
inference/depth_anything_v2.py	Loads/predicts with DAV2 Large, CUDA/autocast, tiling and OOM CPU retry; called by factory; RGB -> raw prediction; prevents unrecorded fallback; real smoke; read load then predict.
inference/conventions.py	Declares inverse-depth/depth direction and robust global normalization; called after prediction; raw -> canonical relative; prevents roof-as-hole inversion; regression tests; read to_relative_surface.
inference/tiling.py	Plans bounded overlapping tiles, fits positive affine overlap and feathers; called by real adapter; large RGB -> blended raw field; prevents gaps/unbounded tiles/seams; tiling tests; read starts then blend loop.
inference/deterministic.py	Fast predictable offline test field; selected only behind safety flag; image -> deterministic raw result; prevents tests downloading GB model; adapter/API tests; never present it as production AI.

11.3 Frontend

File	Atlas
frontend/src/main.tsx	React bootstrap; Vite calls it; mounts App; HTML root -> UI; prevents scattered entry logic; build; read all 5 lines.
types.ts	TypeScript mirror of backend contract; imported by App/viewer/API; JSON -> typed objects; prevents field-name drift; build/tests; start with JobManifest.
api.ts	Fetch/multipart helpers; called by App; calls FastAPI routes; UI values -> requests/JSON; prevents duplicated URL/form logic; App tests; read create/get/calibrate/artifact helpers.
App.tsx	Workflow, uploads, polling, states, evidence, calibration and measurement display; calls API and viewer; user actions -> result workspace; prevents unavailable artifacts appearing active; App tests/live browser; read state, submit, polling, result sections.
SurfaceViewer.tsx	Three.js scene, controls, mesh loading, raycasting and UI toolbar; called by App; grid/texture URLs -> interactive canvas/A-B points; cleanup prevents WebGL leaks; viewer tests/live browser; read setup, raycast, first-person, cleanup.
surfaceGeometry.ts	Pure geometry/colors/measurement helpers; called by viewer; grids -> BufferGeometry and samples; prevents coordinate inversion and display-height measurements; viewer tests; start with sampleMeasurementValue then createSurface.
styles.css	Design tokens, layout and responsive breakpoints; loaded by main; class names -> visual hierarchy; prevents unusable narrow layout; build/live screenshots; start with variables and breakpoints near lines 215/226.
App.test.tsx	UI scientific contract tests; Vitest runs it; mocked API -> rendered states/actions; prevents labels/controls drifting; 10-test frontend total; read test names first.
SurfaceViewer.test.ts	Pure orientation, sampling, distance and structures tests; Vitest runs it; arrays -> expected geometry; prevents axis/height mistakes; read all cases.

11.4 Backend tests and verification scripts

File	What it owns
tests/test_api.py	End-to-end relative workflow, artifacts/hashes, invalid/oversized input, memory budget, capability contract, safe deletion and prebuilt UI.
tests/test_geotiff.py	CRS/transform preservation while vertical state remains locked.
tests/test_terrain.py	DEM success, alignment report/GLB, insufficient overlap and non-metre rejection.
tests/test_calibration.py	Reference pass, exact-grid rejection, poor-evidence lock, independent GCP pass and NoData.

File	What it owns
tests/test_artifacts.py	Orientation, embedded texture/UV/normals/PBR, raised building and display cleanup immutability.
tests/test_adapter_contract.py	Determinism, direction, constant field and test-adapter interlock.
tests/test_tiling.py	Last-pixel coverage, asymmetric orientation, tile cap and affine overlap alignment.
tests/test_evaluation.py	Hand-calculated metrics, masks, insufficient evidence and geospatial matching.
scripts/verify.ps1	Orchestrates fast/full verification and records failure.
smoke_terrain_workflow.py	Runs the packaged DEM path and checks state, ranges and 15 artifacts.
smoke_final_workflow.py	Runs the complete synthetic calibration oracle and checks 18 hashes.
smoke_real_api.py / smoke_real_model.py	Prove the real adapter/checkpoint/device rather than the deterministic test adapter.
smoke_tiled_model.py	Proves the real large-image tiled path executes.
benchmark_height_model.py	Computes real metrics only on aligned reference truth and writes a metric card.
compare_model_variants.py	Compares checkpoint variants only on matched independent evidence.
create_terrain_demo.py / create_calibration_demo.py	Recreate synthetic fixtures with metadata; they are software demos, not acquired truth.

12. Runtime artifact atlas

Artifact	Contains / created by / consumed by	Never mistake it for
raw_model_output.npy	Untouched float model field; pipeline.py + artifacts.py; audit/calibration diagnostics	metres, probability or display-cleaned height
relative_surface.npy	Canonical float32 0-to-1 surface; conventions.py/artifacts.py; calibration and numeric download	absolute DSM
relative_preview.png	Colorized 8-bit human preview; artifact writer; UI comparison	numeric truth or training label
texture.png	Safe RGB source copy, embedded/capped for GLB; artifact writer; viewer/GLB	a height map
relative_height_16bit.png	16-bit relative height encoding; artifact writer; external visualization	metres without scale/offset
relative_grid.json	Downsampled canonical numeric grid and metadata; artifact writer; viewer analysis	full-resolution array
display_grid.json	Downsampled display-cleaned surface; artifact writer; mesh	calibration/measurement truth
reconstructed_structures.json	Convex candidates for visual extrusions; artifact writer; viewer	verified building footprints/heights
relative_surface.glb	Portable indexed textured height mesh with normals/UV/material; artifact writer; external viewers	survey mesh or semantic city model
height_diagnostics.json	Plane-fit/global-tilt and cleanup reporting; artifact writer; evidence panel	an automatic correction or accuracy score

Artifact	Contains / created by / consumed by	Never mistake it for
job.json	Mutable per-job state/progress/config/warnings/artifact list; job store; polling	immutable release manifest
job_manifest.json	Provenance snapshot, model/input metadata and hashes of other artifacts; artifact writer; audit UI	a self-hash or proof of accuracy
aligned_source_dem.npy	Source DEM resampled on optical grid, in metres/NoData; terrain worker; metric export/analysis	higher-resolution evidence than source DEM
metric_surface.npy	Full-resolution metre array from source DEM or passing calibration; terrain/calibration; scientific analysis	proof that upstream truth/model is accurate
metric_analysis_grid.json	Viewer-size metre sample grid; terrain/calibration; A/B clicks	full-resolution surface
metric_surface.tif	CRS/affine/NoData/metre-tagged GeoTIFF; terrain or pass-only calibration; GIS	automatically survey-grade DSM
terrain_alignment_report.json	Source/output CRS, transforms, resolution, overlap, resampling, source and datum; terrain worker; evidence UI	a quality certificate for source DEM
calibration_reference.tif	Exact evidence raster copied into the job; calibration worker; audit	independent truth unless provenance demonstrates it
calibration_report.json	fit, gates, held-out metrics, pass/rejection reasons; calibration worker; UI	model benchmark across unseen landscapes
calibration_error.tif	Candidate metric minus aligned reference residuals in metres; calibration worker; GIS diagnosis	absolute error where reference itself is wrong
calibration_error_preview.png	Colorized residual preview; calibration worker; UI	numeric residual array

All non-manifest artifacts receive SHA-256 digests in the manifest. Hashes detect changed bytes; they do not validate units, provenance or scientific meaning. [Code: `artifacts.py:18-33,735-888` | Evidence: verified 18 hashes | Limitation: integrity is not accuracy | Judge: "Hashes support reproducibility, while calibration and source metadata support meaning."]

13. Security, privacy, licensing and repeatability

- Local bind: start script serves loopback; scene data is not intentionally uploaded to a cloud service. First model/dependency retrieval uses the network.
- Input safety: bounded bytes, pixels and estimated memory; accepted content; corrupt/bomb checks; finite/NoData checks.
- Path safety: opaque validated job IDs, contained artifact resolution and safe deletion rules.
- Privacy limit: local operation is helpful for sensitive scenes, but Windows permissions, logs, caches and user practices still matter.
- Model license: current Large HF checkpoint is documented as CC-BY-NC-4.0; do not claim commercial readiness.
- Data license: every real scene must record provider, product, acquisition date, terms, attribution, CRS, datum, resolution and hashes.
- Google Earth: do not use screenshots/captured output to reconstruct a 3D model. Use only permitted, attributed visual comparison.
- Repeatability: record exact source/model revision, environment, inputs, settings, artifacts, hashes and test output.

14. Verification record for this atlas

Check	Audited result
Packaged backend	40 passed, 1 dependency warning, 3.87 s
Frontend	2 files, 10 tests passed, 22.16 s
Production frontend build	passed; 23 modules; chunk-size advisory only
DEM Terrain smoke	PASS; 640x480; about 841.2 to 5962.5 m; 15 artifacts
Real Photo AI + calibration	PASS on CUDA; DAV2 Large revision 7581137eff8d4e94f6e796d3baea0e9fa79b22d2; 18 hashes; synthetic oracle gate passed
Live browser	both workflows, calibration pass, downloads/viewer controls; no console errors
Source/release comparison	selected backend, frontend and prebuilt entry hashes matched the inspected source release

This proves the documented software path on the audited computer. It does not establish model accuracy on independent real scenes.

15. Judge demonstration and wording

15.1 Recommended 6-minute sequence

1. Say the two-workflow truth in 25 seconds.
2. Run DEM Terrain with the bundled pair or, preferably, a rehearsed licensed real pair.
3. Show Photo vs Height colors, wireframe, movable light and A/B ridge/valley measurement.
4. Open the alignment report, metric GeoTIFF and manifest. State source/datum and native resolutions.
5. Switch briefly to Photo AI and point to `relative` - not `metres`, real model revision and limitation warning.
6. Show the synthetic calibration pass and immediately call it a software oracle.
7. Finish with the blocked real metric card and the staged DFC23 training/evaluation plan.

15.2 Never say

- "A Google Earth screenshot gives us true mountain geometry."
- "The AI knows the metres from one image."
- "Resampling increased DEM resolution."
- "The bundled RMSE proves accuracy."
- "Structures are detected buildings."
- "This predicts disasters" or "This is survey-grade."

15.3 Defensible closing

"TerraFly's current strength is not a magical screenshot-to-truth claim. It is a complete, local and inspectable pipeline that separates relative AI shape, georeferencing, supplied elevation and calibrated metres; records the evidence; refuses metric artifacts when validation fails; and already produces a strong measured-mountain experience from a named DEM."

16. Known inconsistencies and next decisions

See `docs/ownership/INCONSISTENCIES_AND_LIMITATIONS.md` and `docs/ownership/CLAIMS_LEDGER.md`. The most important are the missing independent real Photo AI benchmark, the one-time internet/setup requirement, the older architecture diagram omitting `Metric Source DEM`, the GCP API/UI mismatch and the difference between a downsampled viewer mesh and full-resolution scientific arrays.

17. Recreate TerraFly in the right order

1. Define the states and the rule that no ordinary image produces metres.
2. Implement safe input/raster inspection and a job store.
3. Implement the deterministic test adapter and relative artifact contract.
4. Add the real DAV2 adapter with raw preservation, direction regression and device metadata.
5. Add bounded tiling and tests.
6. Add textured GLB and viewer with separate photo/height/measurement grids.
7. Add DEM Terrain with explicit source/datum and alignment report.
8. Add calibration only with separated controls/validation and fail-closed artifacts.
9. Add release scripts, hashes, live browser checks and honest documentation.
10. Acquire independent evidence before publishing any real accuracy number.

The source-and-line map for every step is `docs/ownership/SOURCE_LINE_INDEX.md`.

TerraFly claims ledger

This ledger separates what was demonstrated on the packaged build from what remains conditional. A judge claim is permitted only at the level shown here.

Status	Claim	Evidence	Limitation	Judge wording
PROVED	The packaged backend passes its automated suite.	40 tests passed on 2026-08-28; backend/tests/	Tests use synthetic fixtures for scientific values.	"The packaged backend passes 40 contract, security, terrain, calibration and artifact tests."
PROVED	The packaged frontend builds and passes component/geometry tests.	2 Vitest files, 10 tests; production build, frontend/src/*.test.*	Browser automation cannot prove every GPU/browser combination.	"The interface and measurement geometry pass 10 automated tests and a production build."
PROVED	DEM Terrain preserves supplied metric elevations through alignment and export.	terrain.py:84-192, 213-348; test_terrain.py:65-137; live DEM smoke	Accuracy is inherited from the source DEM; resampling adds no information.	"TerraFly aligns and visualizes a named DEM without claiming that resampling improves its native accuracy."
PROVED	Photo AI uses the real Depth Anything V2 Large checkpoint on CUDA when available.	depth_anything_v2.py:11-158; verified revision 7581137eff8d4e94f6e796d3baea0e9fa79b22d2, device CUDA	It is a general relative-depth model, not an overhead height model.	"Our current AI baseline is the real DAV2 Large checkpoint, recorded by exact revision; its output remains relative until calibrated."
PROVED	Raw output and canonical normalized surface are preserved separately.	pipeline.py:44-76; artifacts.py:735-888; verified raw_model_output.npy and relative_surface.npy	Neither array contains metres by itself.	"We preserve the model field before conversion and a separate normalized 0-to-1 surface for auditability."
PROVED	The historical roof-as-hole direction error is repaired and regression-tested.	conventions.py:9-98; test_adapter_contract.py:24-55; test_artifacts.py:130-166	Correct local ordering does not prove correct absolute height.	"The adapter declares the model convention and normalizes once; a raised-roof regression fails if height is inverted."
PROVED	Tiled inference is bounded, aligns local scale/offset and feather blends.	tiling.py:8-99; test_tiling.py:9-64	Tiling improves memory/detail handling, not model accuracy evidence.	"Large images use bounded overlapping tiles with affine overlap alignment and one final normalization."
PROVED	Metric Photo AI artifacts are locked behind held-out checks.	calibration.py:136-163, 276-532; test_calibration.py:68-241	A global affine fit cannot correct local domain errors; provenance of user evidence still matters.	"Metric export appears only after independent controls and held-out validation satisfy all documented gates."
PROVED	The bundled calibration demo exercises the complete pass path.	verified CUDA calibration smoke: scale about 40, offset about 100, RMSE about 0.000002 m, 18 artifact hashes	The reference is derived from the output and is not real accuracy evidence.	"This is a synthetic software oracle that proves the gate and artifact path, not the model's accuracy."
PROVED	GLB output contains a texture, UVs, normals and lit material.	artifacts.py:83-296; test_artifacts.py:31-128	It is a height field; steep faces lack real facade pixels.	"The export is a portable textured height mesh with computed normals; neutral steep faces avoid inventing facade texture."
PROVED	The viewer supports orbit, first-person, photo/height modes, wireframe, sun and A/B sampling.	SurfaceViewer.tsx:45-440; surfaceGeometry.ts:122-255; live screenshots	First-person has no collision/terrain-following and the mesh is downsampled.	"The browser provides interactive inspection and point comparison, not a flight simulator or survey instrument."
PROVED	Inputs and artifacts are bounded and path-contained.	imaging.py:31-175; jobs.py:20-26; main.py:206-231; test_api.py:81-131	Local-only operation reduces exposure but does not replace a production security review.	"Uploads, job IDs and artifact paths are validated and the service binds locally."
PARTIAL	TerraFly addresses the SIH single-view height goal.	Photo AI relative path plus reference/GCP calibration	No independent multi-landscape Photo AI benchmark; coarse DEM fusion is not implemented as a learned absolute DSM method.	"We implemented the full evidence-aware pipeline and calibration gate; real single-view height accuracy is the next validated milestone."

Stat us	Claim	Evidence	Limitation	Judge wording
PA RTI AL	TerraFly works offline.	Local runtime and prebuilt UI work after setup/cache	First setup and uncached model download require internet; the release omits model weights and environments.	"After one-time dependency and model setup, the demonstrated workflow runs locally without sending scene data to a cloud service."
PA RTI AL	TerraFly supports real mountains.	DEM Terrain accepts licensed, overlapping optical GeoTIFF + metre DEM	The bundled hero pair is synthetic; a real Sentinel-2/SRTM judge pair must be prepared and attributed.	"Mountains are reconstructed from the supplied DEM evidence; today we are showing the software fixture unless the real pair and attribution are visible."
PA RTI AL	The Structures layer makes cities clearer.	artifacts.py:590-732; surfaceGeometry.ts:66-105	Candidates are nonsemantic and may be trees, missed roofs or false positives.	"Structures is an optional visual aid and never changes the scientific surface or measurements."
BL OC KE D	Photo AI RMSE/MAE/correlation on real remote-sensing scenes.	docs/MODEL_METRIC_CARD.md:1-27; evaluator exists in evaluation.py:9-50	No accepted independent aligned RGB-DSM/nDSM benchmark in the release.	"The real metric card is intentionally blocked until licensed held-out truth is acquired."
BL OC KE D	Survey-grade or high-precision height.	no evidence	General monocular model, affine calibration and source DEM limits	Never claim it.
BL OC KE D	Disaster prediction or operational deployment.	no hazard model or operational validation	Terrain is only one input to hazard analysis.	"TerraFly can supply an elevation inspection foundation; hazard prediction needs separate models, data and validation."
BL OC KE D	Google Earth screenshot reconstruction.	docs/JUDGE_QA.md:19-25; Google Geo Guidelines	Screenshot lacks height metadata; Google terms restrict 3D reconstruction.	"We do not use Google Earth imagery to reconstruct TerraFly geometry."
FU TU RE	Overhead-specific building mask, nDSM and uncertainty model.	docs/TRAINING_ROADMAP.md:17-69	Requires licensed aligned data and geographic splits.	"This is the next separately versioned research stage, not a feature in today's build."
FU TU RE	Optional SAR fusion and multi-city domain adaptation.	DFC23 Track 2 plan	Terms, data audit and RGB baseline must pass first.	"SAR will be an ablation retained only if held-out geographic performance improves."

TerraFly inconsistencies and implementation limitations

No runtime code was changed during this documentation audit. These items should be discussed or repaired in a future engineering task.

Priority	Finding	Evidence	Consequence / recommended follow-up
High	The current Photo AI system has no independent real-world accuracy result.	docs/MODEL_METRIC_CARD.md:1-27; docs/MODEL_UPGRADE_EVALUATION.md; evaluator in evaluation.py:9-50	Keep all accuracy claims blocked. Acquire licensed, aligned RGB-DSM/nDSM truth and run geographic held-out benchmarks.
High	The packaged release is not a zero-install offline bundle.	docs/OPERATOR_GUIDE.md:16-28; scripts/setup.ps1; no packaged .venv, node_modules or model weights	A new judging laptop needs internet and substantial setup. Prepare one already-tested machine and a documented backup video/results bundle.
High	The PS asks for absolute DSM from single optical input using SRTM/GCP scale recovery; the implementation offers either DEM-sourced geometry or global affine calibration.	terrain.py:213-348; calibration.py:348-532; docs/PS_REQUIREMENTS_TRACEABILITY.md	Do not describe DEM Terrain as AI recovery or affine calibration as local-error correction. A coarse-DEM fusion method remains future work.
Medium	docs/ARCHITECTURE.md presents a three-state chain but omits the separate Metric Source DEM state represented by the schema and UI.	docs/ARCHITECTURE.md; schemas.py:9-13; App.tsx:188-288	Use the ownership atlas state diagram as the authoritative explanation; update the older architecture page later.
Medium	The synthetic terrain generator records units=metre as a dataset tag, while live intake warns that no band-unit tag exists.	scripts/create_terrain_demo.py; verified live DEM UI; terrain.py:34-77 reads band units	The numeric values are explicitly treated as metres by the source declaration, but fixture metadata and reader convention should be harmonized.
Medium	"Offline/local" language can be read too broadly.	README.md; docs/OPERATOR_GUIDE.md:16-28; depth_anything_v2.py:11-158	Say "local after one-time setup and model caching". Do not promise air-gapped installation from the source archive.
Medium	GCP calibration exists in the API but the current UI exposes only the reference-raster form.	main.py:186-204; api.ts:3-73; App.tsx:360-406	GCP is an API capability, not a judge-ready click path. Demonstrate reference DSM unless a separate API demo is rehearsed.
Medium	First-person navigation has no collision, gravity or terrain-following.	SurfaceViewer.tsx:318-344	Call it free-flight inspection, not a physically realistic drone simulation.
Medium	The display cleanup and Structures layer can look more regular than the canonical surface.	artifacts.py:299-519,590-732; LIMITATIONS.md	Repeat that cleanup is display-only and structures are nonsemantic. Measurements use analysis/metric grids, not structure extrusion.
Medium	Full-resolution scientific arrays and the downsampled viewer mesh have different roles.	artifacts.py:735-888; SurfaceViewer.tsx:112-260; JUDGE_QA.md:63-65	Do not call the interactive mesh full-resolution survey geometry. Exported arrays/GeoTIFF carry the numeric evidence.
Low	The job manifest cannot hash itself.	artifacts.py:18-33,735-888; verified artifact hash set	Explain that the manifest records all other artifacts; self-hashing would be recursive.
Low	Generated __pycache__, runtime jobs and the production bundle appear in the package inventory beside authored source.	packaged file tree	File-atlas readers can confuse generated files with source. Use FILE_GUIDE.md and the ownership source index.
Low	Historic reports contain earlier milestone test counts as well as final counts.	handoff/DAY_*_TEST_REPORT.md; handoff/FINAL_TEST_REPORT.md; PROJECT_STATUS.md	Quote only the date-stamped final verification: 40 backend tests and 10 frontend tests on 2026-08-28.

Non-negotiable scientific limitations

- A PNG/JPG cannot supply CRS, horizontal scale, vertical units or datum.
- A georeferenced GeoTIFF can still be vertically relative.
- A global positive affine transform cannot repair local nonlinear errors, misregistration or domain shift.
- SRTM/DEM accuracy and resolution remain those of the source provider; reprojection is alignment, not super-resolution.
- Snow, cloud, haze, shadow, forest canopy, water, oblique viewing and global perspective bias can distort Photo AI.
- Near-zero error from the bundled oracle is deliberately excluded from the model metric card.
- No current output proves Himalayan, city-wide, disaster, military, survey-grade or operational accuracy.

TerraFly source and line index

Verified against the packaged project on 2026-08-28. Line numbers refer to the editable source in TerraFly_FINAL_WINDOWS, not generated bundles or cached bytecode.

Runtime and scientific contract

Concern	Primary implementation	Verification evidence
Runtime limits and model configuration	backend/terrafly/config.py:11-31	backend/tests/test_api.py:86-121; live capabilities request
Scientific states and manifest schema	backend/terrafly/schemas.py:9-83	backend/tests/test_api.py:94-112; backend/tests/test_geotiff.py:32-86
Job IDs, atomic state and paths	backend/terrafly/jobs.py:15-69	backend/tests/test_api.py:123-131
Upload limits and image/GeoTIFF inspection	backend/terrafly/imaging.py:15-175	backend/tests/test_api.py:16-92; backend/tests/test_geotiff.py:32-86
API routes and static application	backend/terrafly/main.py:20-239	backend/tests/test_api.py:16-143; live browser run
Photo AI worker	backend/terrafly/pipeline.py:13-96	backend/tests/test_api.py:16-79; real CUDA smoke run
DEM Terrain worker	backend/terrafly/terrain.py:34-348	backend/tests/test_terrain.py:65-176; terrain smoke run
Calibration worker and gates	backend/terrafly/calibration.py:18-532	backend/tests/test_calibration.py:68-241; real calibration smoke run
Metric evaluation	backend/terrafly/evaluation.py:9-50	backend/tests/test_evaluation.py:10-54

AI inference

Concern	Primary implementation	Verification evidence
Adapter result contract	backend/terrafly/inference/base.py:9-19	backend/tests/test_adapter_contract.py:12-65
Direction and one-time normalization	backend/terrafly/inference/conventions.py:9-98	backend/tests/test_adapter_contract.py:24-55; backend/tests/test_artifacts.py:130-166
Real DAV2 Large adapter, CUDA and CPU fallback	backend/terrafly/inference/depth_anything_v2.py:11-158	scripts/smoke_real_model.py; scripts/smoke_real_api.py; verified CUDA run
Adapter selection and test-only lock	backend/terrafly/inference/factory.py:9-23	backend/tests/test_adapter_contract.py:57-65
Deterministic test adapter	backend/terrafly/inference/deterministic.py:9-41	backend/tests/test_adapter_contract.py:12-22
Bounded tiling, overlap alignment and blending	backend/terrafly/inference/tiling.py:8-99	backend/tests/test_tiling.py:9-64; scripts/smoke_tiled_model.py

Artifacts and 3D

Concern	Primary implementation	Verification evidence
Hashing and color map	backend/terrafly/artifacts.py:18-45	backend/tests/test_api.py:16-79; smoke hash verification

Concern	Primary implementation	Verification evidence
Train/evaluation spatial split helper	backend/terrafly/artifacts.py:56-80	backend/tests/test_calibration.py:68-140
Vertex normals	backend/terrafly/artifacts.py:83-98	backend/tests/test_artifacts.py:31-128
Textured GLB writer	backend/terrafly/artifacts.py:101-296	backend/tests/test_artifacts.py:31-166
Display-only cleanup	backend/terrafly/artifacts.py:299-519	backend/tests/test_artifacts.py:168-210
Tilt diagnostics	backend/terrafly/artifacts.py:522-560	height_diagnostics.json in verified real-model job
Optional nonsemantic structures	backend/terrafly/artifacts.py:590-732	reconstructed_structures.json; frontend geometry tests
Artifact bundle writer	backend/terrafly/artifacts.py:735-888	manifest and 18 verified artifact hashes

Frontend

Concern	Primary implementation	Verification evidence
API helpers and multipart forms	frontend/src/api.ts:3-73	frontend/src/App.test.tsx:10-190
Shared API types	frontend/src/types.ts:1-54	TypeScript build and Vitest
Workflow selection, upload, polling and evidence UI	frontend/src/App.tsx:41-501	frontend/src/App.test.tsx:10-190; live browser run
3D scene, controls, raycasting and cleanup	frontend/src/SurfaceViewer.tsx:18-440	frontend/src/SurfaceViewer.test.ts:15-142; live browser run
Mesh, colors, structures and measurements	frontend/src/surfaceGeometry.ts:3-255	frontend/src/SurfaceViewer.test.ts:15-142
Responsive visual system	frontend/src/styles.css:1-239	production build; live screenshots at desktop viewport
React entry point	frontend/src/main.tsx:1-5	production build

Launch, verification and demonstration

File	Main responsibility	Evidence
Start-TerraFly.cmd:1-6	Windows double-click entry point	live packaged launch
scripts/start.ps1:1-115	Environment checks, port ownership, local server and browser	live packaged launch at 127.0.0.1
scripts/setup.ps1:1-140	One-time Python, frontend and model setup	operator guide documents network requirement
scripts/verify.ps1:1-110	Fast/full release verification	fast and full smoke scripts
scripts/smoke_terrain_workflow.py:1-154	End-to-end DEM Terrain evidence	verified PASS, 15 artifacts
scripts/smoke_final_workflow.py:1-205	Synthetic calibration oracle	verified PASS, 18 hashes
scripts/smoke_real_api.py:1-166	Real model API and artifact check	verified CUDA run
scripts/smoke_real_model.py:1-86	Direct real adapter check	model revision/device evidence

File	Main responsibility	Evidence
scripts/smoke_tiled_model.py:1-129	Real tiled inference	tiling smoke contract
scripts/benchmark_height_model.py:1-211	Independent reference evaluation	docs/MODEL_METRIC_CARD.md:1-27 remains BLOCKED
scripts/compare_model_variants.py:1-185	Checkpoint comparison on matched truth	no accuracy claim without data
scripts/create_terrain_demo.py:1-155	Reproducible synthetic terrain fixture	sample_data/terrafly_terrain_demo_metadata.json
scripts/create_calibration_demo.py:1-172	Reproducible software oracle fixture	sample_data/terrafly_calibration_demo_metadata.json

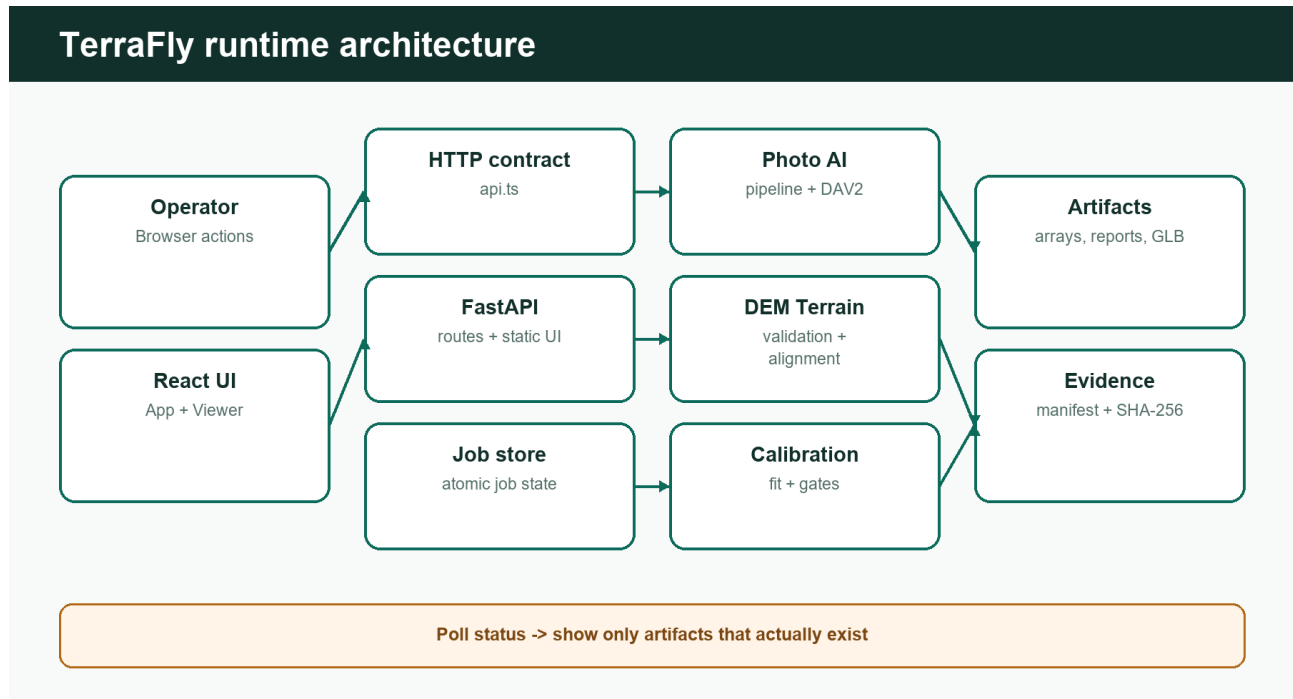
Reading order for a complete beginner

1. README.md:3-101 and docs/OPERATOR_GUIDE.md:1-175.
2. backend/terrafly/schemas.py:9-83 to learn the states and job vocabulary.
3. frontend/src/api.ts:3-73, then backend/terrafly/main.py:20-239 to see the browser-to-server contract.
4. backend/terrafly/terrain.py:34-348 and backend/terrafly/pipeline.py:18-96 as the two workers.
5. backend/terrafly/calibration.py:136-163, 276-532 for the evidence gate.
6. backend/terrafly/artifacts.py:101-296, 735-888 and frontend/src/SurfaceViewer.tsx:112-440 for 3D and outputs.
7. Read the matching test beside every subsystem before changing it.

Visual evidence and editable diagrams

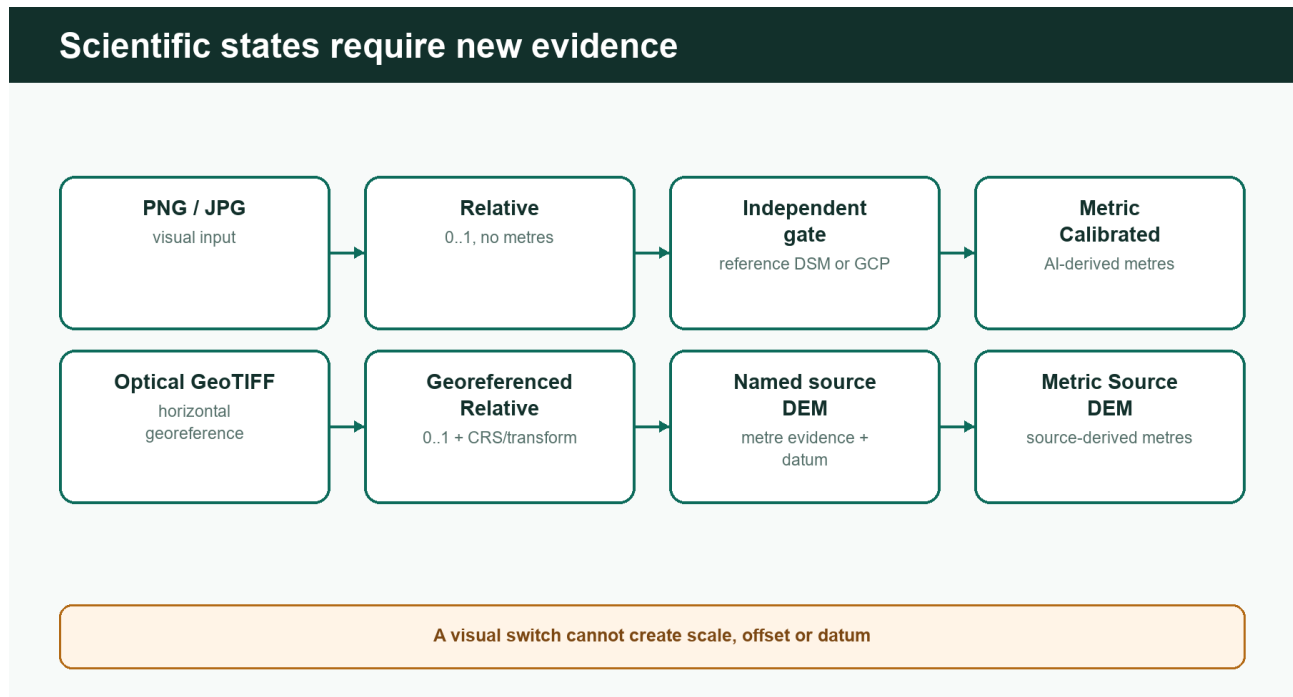
The diagram sources are editable Mermaid files in docs/ownership/diagrams. Screenshot SVG overlays remain editable; annotated PNGs are flattened copies for portable viewing. Original captures are preserved.

TerraFly runtime architecture



Poll status -> show only artifacts that actually exist

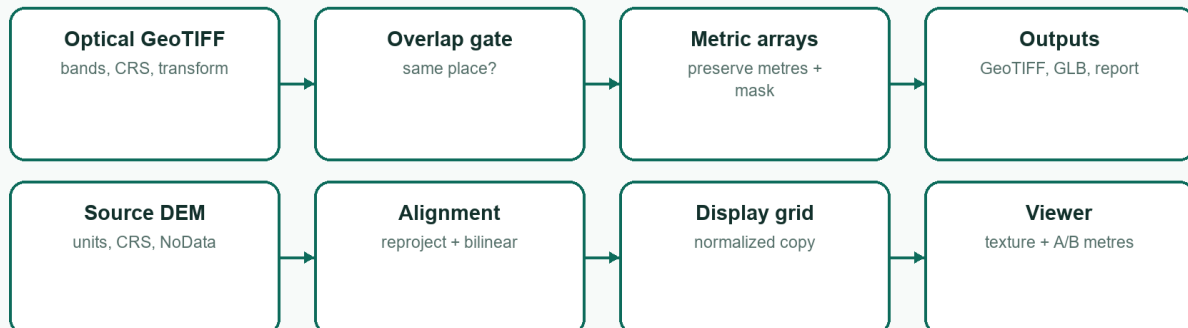
Scientific states require new evidence



A visual switch cannot create scale, offset or datum

DEM Terrain workflow

DEM Terrain workflow

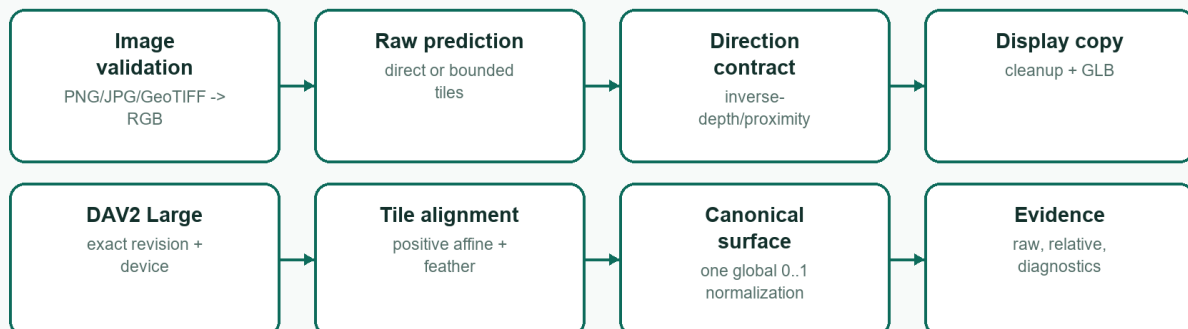


Resampling aligns the grid; it never upgrades native DEM information

Resampling aligns the grid; it never upgrades native DEM information

Photo AI workflow

Photo AI workflow

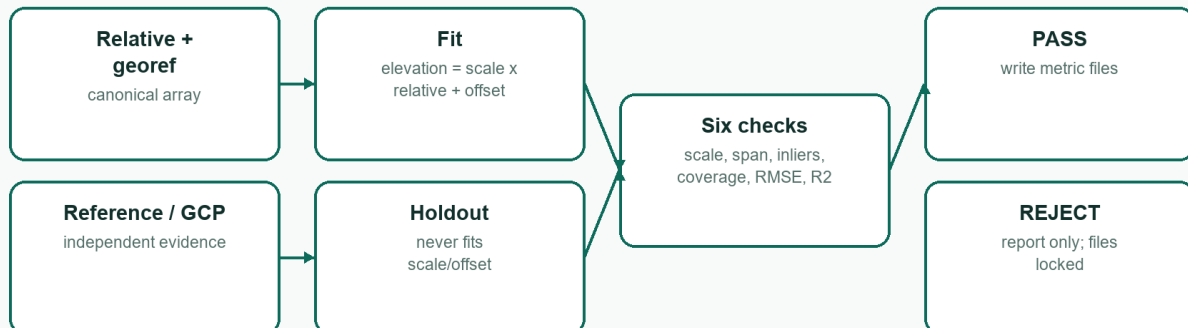


The first output remains relative even when it looks three-dimensional

The first output remains relative even when it looks three-dimensional

Metric calibration gate

Metric calibration gate

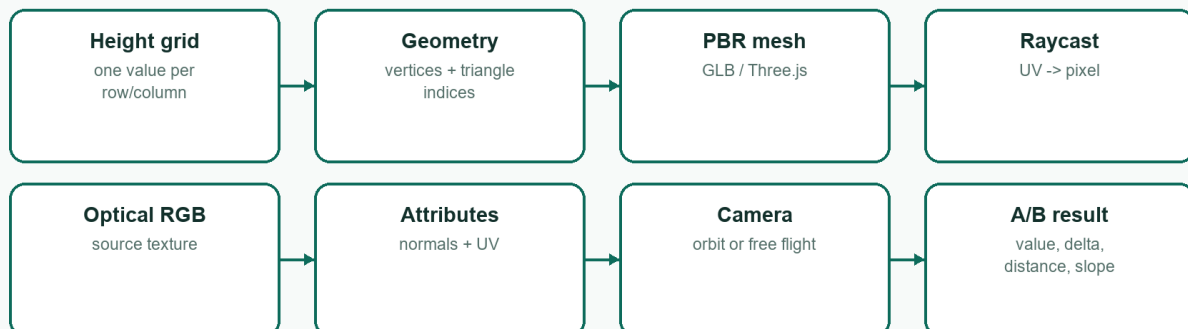


Rejection is a correct scientific result, not a software failure

Rejection is a correct scientific result, not a software failure

Height pixels to textured mesh and measurement

Height pixels to textured mesh and measurement

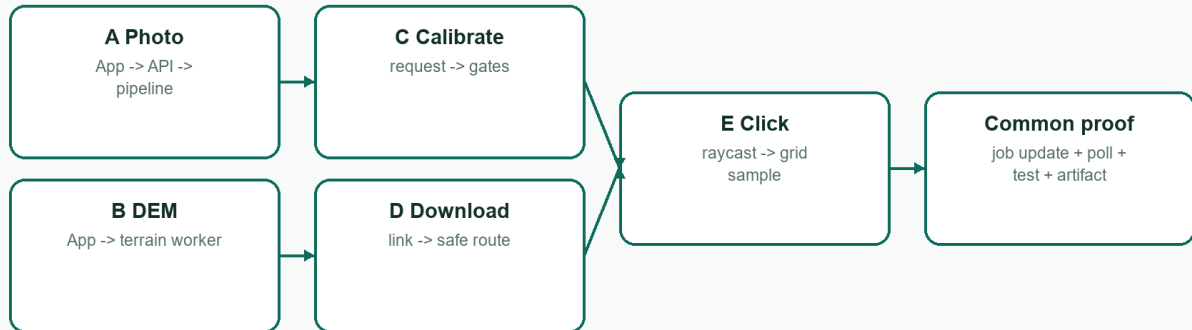


Display exaggeration changes vertices, never stored measurements

Display exaggeration changes vertices, never stored measurements

Five auditable request flows

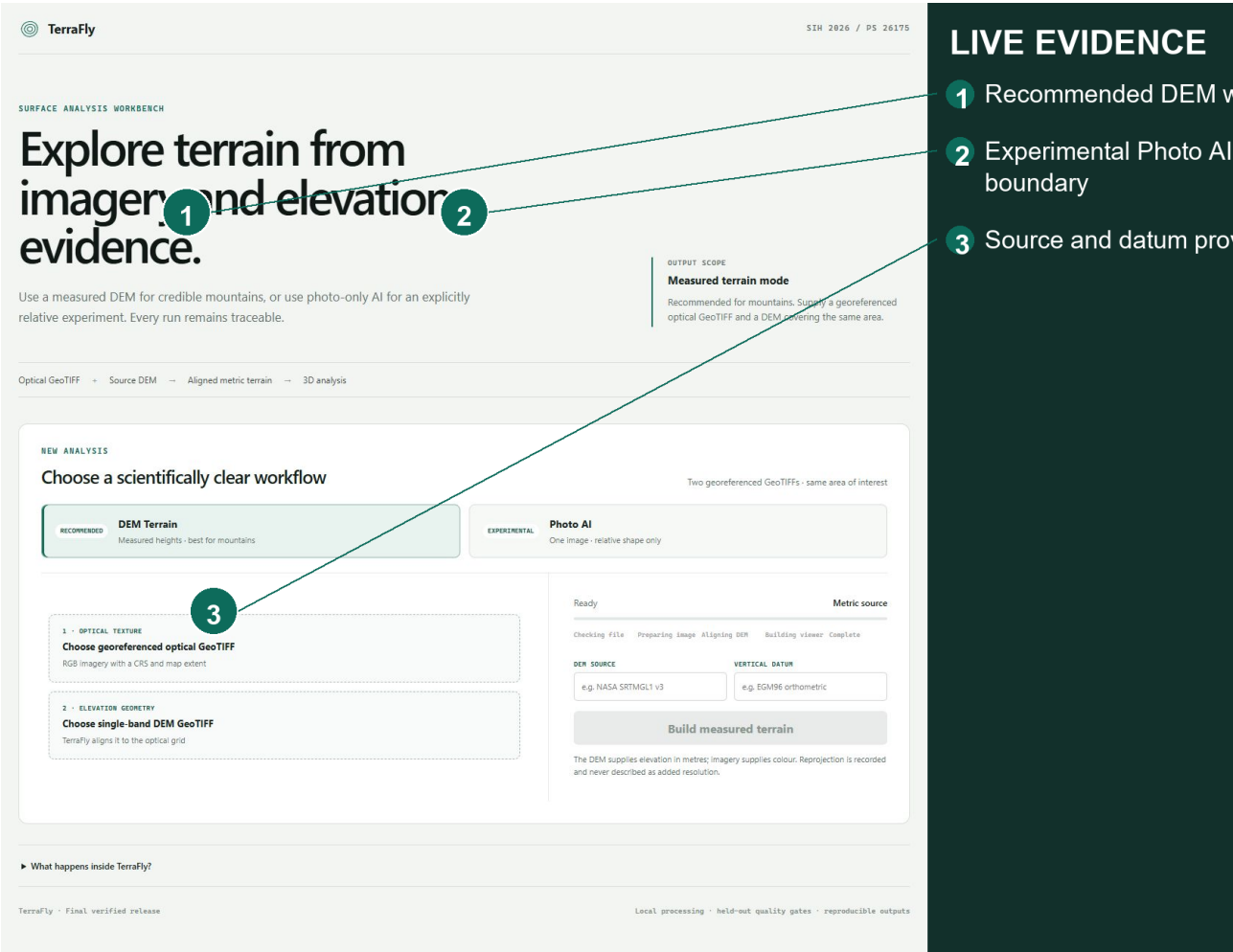
Five auditable request flows



Every visible result has a route, worker, stored evidence and supporting test

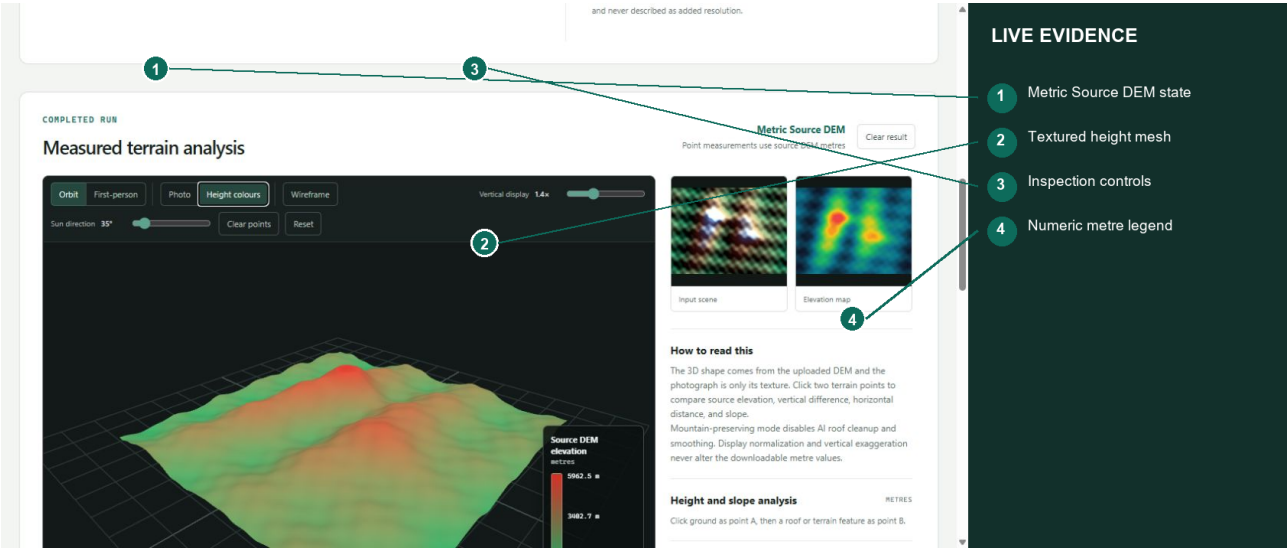
Every visible result has a route, worker, stored evidence and supporting test

01 Live Start Screen



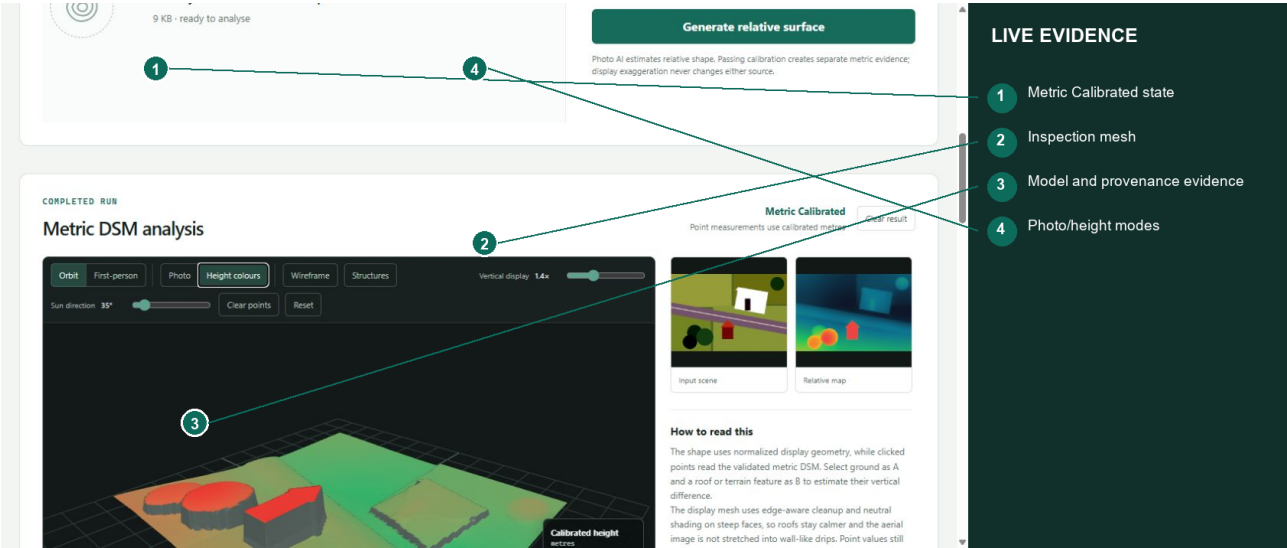
Actual live application capture with editable SVG callouts stored beside this image.

02 Live Dem Terrain Result



Actual live application capture with editable SVG callouts stored beside this image.

03 Live Photo Ai Calibrated



Actual live application capture with editable SVG callouts stored beside this image.

04 Live Calibration Passed

The screenshot displays the TerraFly application interface. At the top left, a 3D terrain model is shown with a color-coded height scale (0 to 100 meters). Below it, a 'Metric calibration' section is visible, featuring input fields for 'Input CRS' (EPSG:32643), 'Input grid' (480 x 320 pixels), and a 'Viewer' section. The 'Viewer' section contains the text: 'Normalized display geometry + calibrated metric point analysis after a pass'. To the right of the 'Metric calibration' section, there is a 'Reference DSM - GeoTIFF' field with the value 'terrafly_calibration_dsmo_reference.tif'. Below this, there is a 'Bundled synthetic software oracle' field. Further down, there is a 'Vertical datum' field with the value 'Demo benchmark datum' and a 'Maximum held-out RMSE - m' field with the value '0.5'. At the bottom of this section is a green button labeled 'Evaluate and calibrate'. On the right side of the interface, a 'LIVE EVIDENCE' sidebar is visible, containing three items: '1 Gate status: Passed', '2 Report and metric artifacts', and '3 Synthetic oracle - not accuracy'. Three numbered callouts (1, 2, 3) are overlaid on the image, pointing to these items in the sidebar.

1 Gate status: Passed

2 Report and metric artifacts

3 Synthetic oracle - not accuracy

Actual live application capture with editable SVG callouts stored beside this image.